TOWER OF HANOI

A Recursive Program to Solve the Tower of Hanoi Problem for n Disks

Name	**:**	Rudra Pratap Singh
SAP ID	**:**	590032811
Batch	**:**	16

1. Problem Statement

Write a recursive program to solve the Tower of Hanoi problem for n disks and display all the moves.

2. Aim

To write a C program that uses recursion to solve the Tower of Hanoi puzzle for any given number of disks n, and to print every move required to transfer all the disks from the source rod to the destination rod.

3. Introduction

The Tower of Hanoi is a classic mathematical puzzle. It consists of three rods and n disks of different sizes, which can slide onto any rod. The puzzle starts with all the disks stacked on one rod in order of size, with the smallest disk at the top and the largest at the bottom.

The objective is to move the entire stack to another rod while obeying the following rules:

1. Only one disk can be moved at a time.
2. Only the topmost disk of a rod can be picked up and moved.
3. A larger disk can never be placed on top of a smaller disk.

In this project the three rods are named **A** (source), **B** (auxiliary) and **C** (destination).

4. Recursive Approach

The problem has a natural recursive structure. To move n disks from the source rod to the destination rod using an auxiliary rod:

1. Move the top **n − 1** disks from the source to the auxiliary rod (using the destination rod as helper).
2. Move the largest disk (**disk n**) from the source to the destination rod.
3. Move the **n − 1** disks from the auxiliary rod to the destination rod (using the source rod as helper).

Base case: when n = 1, simply move the single disk directly from the source to the destination.

5. Algorithm

Function: `towerOfHanoi(n, source, auxiliary, destination)`

1. Start.
2. If n = 1, print “Move disk 1 from source to destination” and return.
3. Call towerOfHanoi(n − 1, source, destination, auxiliary).
4. Print “Move disk n from source to destination”.
5. Call towerOfHanoi(n − 1, auxiliary, source, destination).

6. Stop.

Main program: read the number of disks n from the user, validate that n ≥ 1, call towerOfHanoi(n, 'A', 'B', 'C') and display the total number of moves.

6. Source Code

Language: C **File name:** `hanoi.c`

```c
#include <stdio.h>

/* Global counter to keep track of the number of moves */
int moveCount = 0;

/* Recursive function to solve Tower of Hanoi */
void towerOfHanoi(int n, char source, char auxiliary, char destination)
{
    /* Base case: only one disk left to move */
    if (n == 1)
    {
        moveCount++;
        printf("Move %d: Move disk 1 from %c to %c\n",
               moveCount, source, destination);
        return;
    }

    /* Step 1: Move n-1 disks from source to auxiliary using destination */
    towerOfHanoi(n - 1, source, destination, auxiliary);

    /* Step 2: Move the largest disk from source to destination */
    moveCount++;
    printf("Move %d: Move disk %d from %c to %c\n",
           moveCount, n, source, destination);

    /* Step 3: Move n-1 disks from auxiliary to destination using source */
    towerOfHanoi(n - 1, auxiliary, source, destination);
}

int main()
{
    int n;

    printf("Enter the number of disks: ");
    scanf("%d", &n);

    if (n < 1)
    {
        printf("Number of disks must be at least 1.\n");
        return 1;
    }

    printf("\nSteps to solve Tower of Hanoi with %d disk(s):\n\n", n);
    towerOfHanoi(n, 'A', 'B', 'C');

    printf("\nTotal number of moves = %d\n", moveCount);
    return 0;
}
```

7. Explanation of the Code

- `moveCount` – a global variable that counts and numbers every move made.
- `towerOfHanoi()` – the recursive function. It takes the number of disks and the names of the three rods as parameters.
- Base case (n == 1) – the recursion stops here by moving one disk directly and returning.
- Recursive case – the function calls itself twice with n − 1 disks: once to clear the way and once to place the smaller disks on top of the largest disk after it has been moved.
- `main()` – takes input n, checks that it is valid, calls the recursive function and prints the total number of moves.

8. Sample Output

Test Case 1: n = 1

```
Enter the number of disks: 1

Steps to solve Tower of Hanoi with 1 disk(s):

Move 1: Move disk 1 from A to C

Total number of moves = 1
```

Test Case 2: n = 2

```
Enter the number of disks: 2

Steps to solve Tower of Hanoi with 2 disk(s):

Move 1: Move disk 1 from A to B
Move 2: Move disk 2 from A to C
Move 3: Move disk 1 from B to C

Total number of moves = 3
```

Test Case 3: n = 3

```
Enter the number of disks: 3

Steps to solve Tower of Hanoi with 3 disk(s):

Move 1: Move disk 1 from A to C
Move 2: Move disk 2 from A to B
Move 3: Move disk 1 from C to B
Move 4: Move disk 3 from A to C
Move 5: Move disk 1 from B to A
Move 6: Move disk 2 from B to C
Move 7: Move disk 1 from A to C

Total number of moves = 7
```

Test Case 4: n = 4

```
Enter the number of disks: 4

Steps to solve Tower of Hanoi with 4 disk(s):

Move 1: Move disk 1 from A to B
Move 2: Move disk 2 from A to C
Move 3: Move disk 1 from B to C
Move 4: Move disk 3 from A to B
Move 5: Move disk 1 from C to A
Move 6: Move disk 2 from C to B
Move 7: Move disk 1 from A to B
Move 8: Move disk 4 from A to C
Move 9: Move disk 1 from B to C
Move 10: Move disk 2 from B to A
Move 11: Move disk 1 from C to A
Move 12: Move disk 3 from B to C
Move 13: Move disk 1 from A to B
Move 14: Move disk 2 from A to C
Move 15: Move disk 1 from B to C

Total number of moves = 15
```

Test Case 5: Invalid input (n = 0)

```
Enter the number of disks: 0
Number of disks must be at least 1.
```

9. Complexity Analysis

Let T(n) be the number of moves needed for n disks. Since the function makes two recursive calls with n − 1 disks and one direct move:

$$T(n) = 2 \cdot T(n-1) + 1, \quad \text{with } T(1) = 1$$

Solving this recurrence gives **$T(n) = 2^n - 1$** moves, which is the minimum possible. The table below verifies this against the program output:

Number of disks (n)	Moves by formula ($2^n - 1$)	Moves by program
1	1	1
2	3	3
3	7	7
4	15	15

- **Time complexity:** $O(2^n)$ – the number of moves doubles with every extra disk.
- **Space complexity:** O(n) – the maximum depth of the recursion call stack is n.

10. Conclusion

The Tower of Hanoi problem was solved successfully using recursion. The program prints every move for any number of disks, and the total number of moves matches the theoretical minimum of $2^n - 1$. The project shows how a large problem can be broken into smaller identical sub-problems, although the exponential growth in moves makes it practical only for a small number of disks.